

Variabili d'Ambiente e Configurazione Avanzata

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Distinguere tra `env[].value` (valore statico), `env[].valueFrom` (valore dinamico) ed `envFrom` (importazione bulk)
 - Configurare `fieldRef` per iniettare metadati del Pod (nome, namespace, IP, labels, annotations) come variabili d'ambiente
 - Configurare `resourceFieldRef` per esporre requests e limits di CPU/memoria come variabili (utile per configurare la JVM)
 - Usare `envFrom` per importare tutte le chiavi di un ConfigMap o Secret con prefisso opzionale
 - Spiegare l'ordine di precedenza e i possibili conflitti tra `env` e `envFrom`
 - Applicare pattern di configurazione avanzata per applicazioni Java/Spring su ARO
-

Teoria e Concetti Chiave

Panoramica dei Meccanismi di Iniezione

Variabile d'ambiente in un container:

— <code>env[].value</code>	→ valore statico hardcoded nel manifest
— <code>env[].valueFrom</code>	
— <code>configMapKeyRef</code>	→ chiave singola da ConfigMap
— <code>secretKeyRef</code>	→ chiave singola da Secret (valore cifrato in transit)
— <code>fieldRef</code>	→ metadato del Pod (nome, namespace, IP, labels...)
— <code>resourceFieldRef</code>	→ valore di requests/limits del container
— <code>envFrom[]</code>	
— <code>configMapRef</code>	→ tutte le chiavi di un ConfigMap
— <code>secretRef</code>	→ tutte le chiavi di un Secret

env[].value — Valore Statico

Il modo più semplice: il valore è scritto direttamente nel manifest YAML.

```
env:
- name: ENVIRONMENT
  value: "production"
- name: API_VERSION
  value: "v2"
```

Limitazioni: il valore è statico e non può fare riferimento ad altri oggetti Kubernetes. Per aggiornarlo occorre modificare il Deployment.

configMapKeyRef e secretKeyRef

Leggono una **singola chiave** da un ConfigMap o Secret:

```
env:
- name: DB_HOST
  valueFrom:
    configMapKeyRef:
      name: db-config      # ← nome del ConfigMap
      key: host             # ← chiave da leggere
      optional: true       # ← se true, Pod si avvia anche se ConfigMap/chiave
assenti
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: password
      optional: false      # ← default: false; Pod non si avvia se assente
```

fieldRef — Metadati del Pod (Downward API)

Permette al container di conoscere informazioni su sé stesso senza chiamare l'API server. Utile per logging strutturato, tracing, identificazione dell'istanza.

Campo <code>fieldPath</code>	Valore iniettato	Use case
<code>metadata.name</code>	Nome del Pod	Identificazione istanza nei log
<code>metadata.namespace</code>	Namespace corrente	Multi-tenant apps
<code>metadata.labels['app']</code>	Valore della label <code>app</code>	Feature flags per ambiente
<code>metadata.annotations['version']</code>	Valore dell'annotation <code>version</code>	Versioning runtime
<code>status.podIP</code>	IP del Pod	Self-registration in service mesh
<code>status.hostIP</code>	IP del nodo	Routing regionale
<code>spec.nodeName</code>	Nome del nodo	Affinità, debug
<code>spec.serviceAccountName</code>	Nome del ServiceAccount	Audit trail

```
env:  
- name: POD_NAME  
  valueFrom:  
    fieldRef:  
      fieldPath: metadata.name           # ← nome del Pod (es. myapp-6b4c9d-xz9kp)  
- name: POD_NAMESPACE  
  valueFrom:  
    fieldRef:  
      fieldPath: metadata.namespace      # ← namespace (es. myproject)  
- name: POD_IP  
  valueFrom:  
    fieldRef:  
      fieldPath: status.podIP           # ← IP del Pod (es. 10.128.2.15)
```

resourceFieldRef — Requests e Limits come Variabili

Espone i valori di CPU e memoria configurati nel container come variabili d'ambiente. Fondamentale per configurare automaticamente la JVM di applicazioni Java:

Campo resource	Valore	Use case
requests.cpu	CPU request (es. 0.5)	Configurazione thread pool
requests.memory	Memoria request in bytes	Heap JVM iniziale (-Xms)
limits.cpu	CPU limit	Configurazione parallelismo
limits.memory	Memoria limit in bytes	Heap JVM massimo (-Xmx)

```
env:
- name: MEMORY_LIMIT
  valueFrom:
    resourceFieldRef:
      containerName: app          # ← nome del container (obbligatorio se
multi-container)
      resource: limits.memory
      divisor: "1Mi"              # ← converti bytes in MiB (default: "1" =
bytes)
- name: CPU_REQUEST
  valueFrom:
    resourceFieldRef:
      containerName: app
      resource: requests.cpu
      divisor: "1m"               # ← converti in millicores
```

envFrom — Importazione Bulk

Importa **tutte le chiavi** di un ConfigMap o Secret come variabili d'ambiente:

```
envFrom:
- configMapRef:
  name: app-config                # ← tutte le chiavi diventano env vars
  optional: false
- prefix: SECRET_                # ← prefisso per evitare conflitti
  secretRef:
  name: app-secrets
```

⚠ Attenzione: Con **envFrom**, le chiavi del ConfigMap/Secret che contengono caratteri non validi per i nomi di variabili d'ambiente (es. **.**, **-**) vengono **silenziosamente ignorate**. Verificare sempre con **oc exec pod -- env | grep PREFIX**.

Ordine di Precedenza e Conflitti

Quando `env` e `envFrom` definiscono lo stesso nome di variabile, vince sempre `env`:

Ordine di override (dal più basso al più alto):

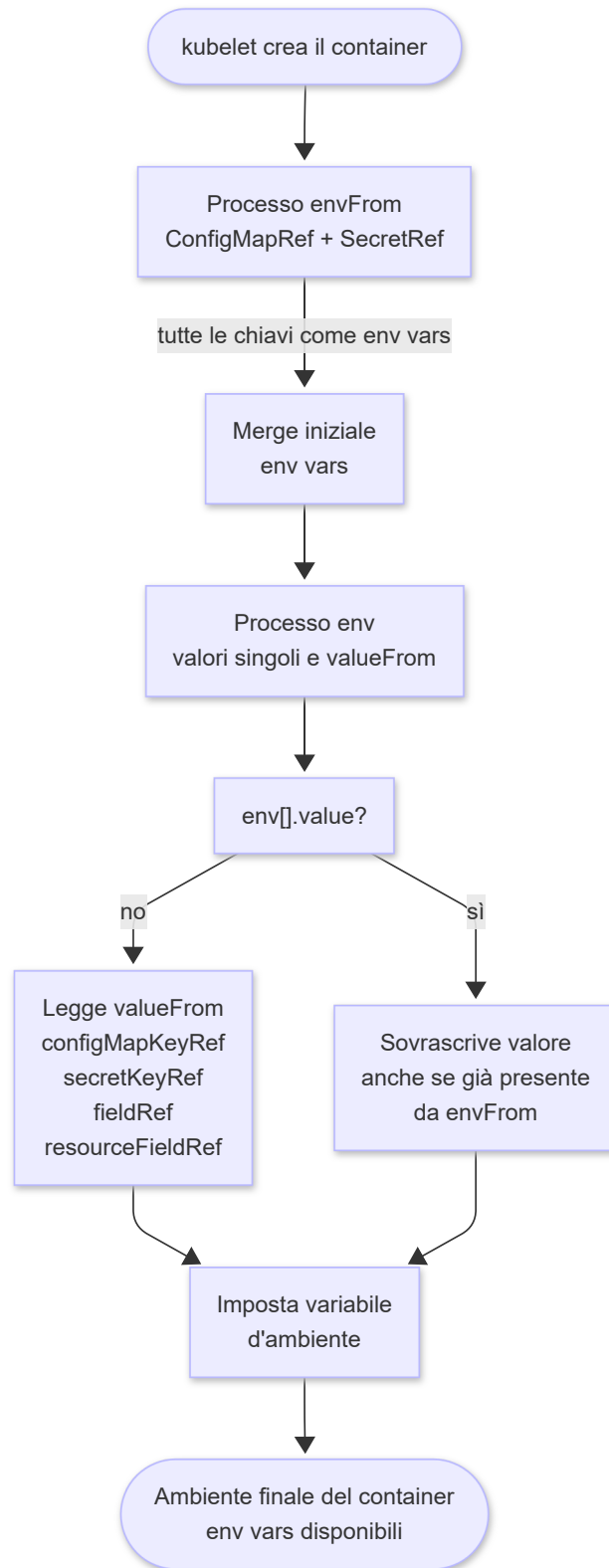
1. `envFrom.configMapRef` (importazione bulk, priorità più bassa)
2. `envFrom.secretRef` (importazione bulk)
3. `env[].valueFrom` (singola chiave da ConfigMap/Secret/fieldRef)
4. `env[].value` (valore statico, priorità più alta)

Esempio di conflitto:

```
envFrom:  
- configMapRef:  
  name: app-config # ← contiene LOG_LEVEL=info  
env:  
- name: LOG_LEVEL  
  value: "debug" # ← questo vince → LOG_LEVEL=debug
```

Diagrammi

Ordine di Risoluzione delle Variabili d'Ambiente



Comandi Pratici con Output Atteso

```
# Creare un Pod con variabili d'ambiente miste per test
oc apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: env-demo
  namespace: myproject
  labels:
    app: env-demo
spec:
  containers:
  - name: demo
    image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
    command: ["sleep", "3600"]
    resources:
      requests:
        memory: "64Mi"
        cpu: "100m"
      limits:
        memory: "128Mi"
        cpu: "500m"
    env:
      - name: POD_NAME
        valueFrom:
          fieldRef:
            fieldPath: metadata.name
      - name: POD_NAMESPACE
        valueFrom:
          fieldRef:
            fieldPath: metadata.namespace
      - name: MEMORY_LIMIT_MIB
        valueFrom:
          resourceFieldRef:
            containerName: demo
            resource: limits.memory
            divisor: "1Mi"
      - name: STATIC_VAR
        value: "hello-aro"
EOF

# Output atteso:
# pod/env-demo created
```

```
# Verificare le variabili d'ambiente Downward API nel container
oc exec env-demo -n myproject -- env | grep -E "POD_|MEMORY_|STATIC_"

# Output atteso:
```

```
# POD_NAME=env-demo
# POD_NAMESPACE=myproject
# MEMORY_LIMIT_MIB=128
# STATIC_VAR=hello-aro
```

```
# Creare un ConfigMap e verificare envFrom con prefisso
oc create configmap envfrom-demo \
  --from-literal=API_URL=https://api.example.com \
  --from-literal=TIMEOUT=30 \
  -n myproject

oc exec env-demo -n myproject -- printenv | grep "APP_"
# (nessun output se il Pod non ha envFrom; aggiungere envFrom in un nuovo Pod)
```

```
# Verificare i valori di resourceFieldRef nel container
oc exec env-demo -n myproject -- sh -c 'echo "Memory limit:
${MEMORY_LIMIT_MIB}Mi"'

# Output atteso:
# Memory limit: 128Mi
```

```
# Vedere tutte le env vars di un Pod (utile per debug)
oc exec deployment/myapp -n myproject -- env | sort

# Output atteso (estratto):
# API_URL=https://api.example.com
# DB_HOST=postgres-svc
# DB_PASSWORD=(valore del Secret)
# ENVIRONMENT=production
# POD_IP=10.128.2.15
# POD_NAME=myapp-6b4c9d-xz9kp
# POD_NAMESPACE=myproject
```

```
# Verificare quale valore vince in caso di conflitto envFrom vs env
oc get pod env-demo -n myproject -o jsonpath='{.spec.containers[0].env}' | python3
-m json.tool

# Output atteso: lista JSON di tutte le env vars definite nel manifest
```

```
# Usare oc set env per aggiungere variabili d'ambiente a un Deployment esistente
oc set env deployment/myapp \
  NEW_FEATURE_FLAG=enabled \
```



```
-n myproject
```

```
# Output atteso:  
# deployment.apps/myapp updated
```

```
# Rimuovere una variabile d'ambiente da un Deployment  
oc set env deployment/myapp \  
  NEW_FEATURE_FLAG- \      # ← il trattino finale rimuove la variabile  
  -n myproject
```

```
# Output atteso:  
# deployment.apps/myapp updated
```

Esempi YAML Completi

Deployment con Configurazione Avanzata Completa

```
# ConfigMap per l'applicazione
apiVersion: v1
kind: ConfigMap
metadata:
  name: myapp-config
  namespace: myproject
data:
  APP_ENV: "production"
  LOG_LEVEL: "info" # ← sarà sovrascritta da env[] se definita
  DB_HOST: "postgres-svc"
  DB_PORT: "5432"
  API_BASE_URL: "https://api.internal.example.com"
---
# Deployment con tutti i meccanismi di iniezione variabili
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  namespace: myproject
  labels:
    app: myapp
    version: "2.1.0"
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
        version: "2.1.0"
    annotations:
      deployment-time: "2026-05-24" # ← accessibile via fieldRef
  spec:
    containers:
      - name: app
        image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
        command: ["/bin/sh", "-c", "env | sort && sleep 3600"]
        resources:
          requests:
            memory: "256Mi" # ← usato da resourceFieldRef
            cpu: "200m"
          limits:
            memory: "512Mi" # ← MEMORY_LIMIT_MIB = 512
            cpu: "1000m"
```

```

# --- envFrom: importa tutte le chiavi del ConfigMap ---
envFrom:
- configMapRef:
    name: myapp-config          # ← APP_ENV, LOG_LEVEL, DB_HOST, DB_PORT,
API_BASE_URL
- prefix: SECRET_              # ← prefisso per evitare conflitti con env
[]

    secretRef:
        name: db-credentials    # ← SECRET_username, SECRET_password

# --- env: valori individuali con precedenza su envFrom ---
env:
# Valore statico (sovrascrive LOG_LEVEL da ConfigMap)
- name: LOG_LEVEL
  value: "warn"                # ← override: questo vince su envFrom

# Downward API: metadati del Pod
- name: POD_NAME
  valueFrom:
    fieldRef:
      fieldPath: metadata.name    # ← nome del Pod specifico
- name: POD_NAMESPACE
  valueFrom:
    fieldRef:
      fieldPath: metadata.namespace # ← namespace
- name: POD_IP
  valueFrom:
    fieldRef:
      fieldPath: status.podIP      # ← IP del Pod
- name: NODE_NAME
  valueFrom:
    fieldRef:
      fieldPath: spec.nodeName     # ← nodo che esegue il Pod

# Downward API: resources del container (utile per JVM tuning)
- name: MEMORY_LIMIT_MIB
  valueFrom:
    resourceFieldRef:
      containerName: app
      resource: limits.memory
      divisor: "1Mi"              # ← converti bytes → MiB → "512"
- name: CPU_REQUEST_MILLICORES
  valueFrom:
    resourceFieldRef:
      containerName: app
      resource: requests.cpu
      divisor: "1m"               # ← converti → millicores → "200"

# Costruito da risorse: JAVA_OPTS calcolato dall'operatore (o da
initContainer)
# Alternativa: usare MEMORY_LIMIT_MIB nelle startup script dell'app
- name: JAVA_TOOL_OPTIONS
  value: "-XX:MaxRAMPercentage=75.0" # ← JVM usa 75% del memory limit

```

```
# Singola chiave da ConfigMap con nome diverso
- name: DATABASE_HOST
  valueFrom:
    configMapKeyRef:
      name: myapp-config
      key: DB_HOST           # ← rinomina la chiave
```

Note Specifiche per Azure ARO

Iniezione di Endpoint Azure tramite fieldRef e ConfigMap

Su ARO, gli endpoint dei servizi Azure (Service Bus, Key Vault, Storage Account) vengono tipicamente iniettati tramite ConfigMap gestiti da pipeline CI/CD o da Azure Service Operator:

```
# Creare un ConfigMap con endpoint Azure specifici dell'ambiente ARO
oc create configmap azure-endpoints \
  --from-literal=AZURE_KEYVAULT_URI=https://prod-kv.vault.azure.net/ \
  --from-literal=AZURE_SERVICEBUS_NAMESPACE=prod-bus.servicebus.windows.net \
  --from-literal=AZURE_STORAGE_ACCOUNT=prodstorageaccount \
  -n myproject
```

Workload Identity e Variabili AZURE_CLIENT_ID

Con **Azure Workload Identity**, le credenziali Azure vengono iniettate automaticamente dal mutating webhook come variabili d'ambiente:

```
# Le seguenti variabili vengono iniettate automaticamente dal webhook Workload Identity
# quando il Pod usa un ServiceAccount annotato:
# AZURE_CLIENT_ID      → Client ID della Managed Identity
# AZURE_TENANT_ID      → Azure Tenant ID
# AZURE_FEDERATED_TOKEN_FILE → path al token OIDC

# Verificare le variabili iniettate da Workload Identity
oc exec deployment/myapp -n myproject -- env | grep AZURE_
# AZURE_CLIENT_ID=<managed-identity-client-id>
# AZURE_TENANT_ID=<tenant-id>
# AZURE_FEDERATED_TOKEN_FILE=/var/run/secrets/azure/tokens/azure-identity-token
```

Pattern JVM Sizing per ARO

Su ARO, le applicazioni Java devono essere configurate per rispettare i limits del container, evitando che la JVM usi più memoria del consentito (causando OOMKilled):

```
env:
# Calcola heap JVM automaticamente dal memory limit del container
- name: JAVA_TOOL_OPTIONS
  value: "-XX:MaxRAMPercentage=75.0 -XX:InitialRAMPercentage=50.0 -XX:+UseContainerSupport"
# Con container support abilitato (default JVM 11+), la JVM legge i cgroup limits
# e si autodimensiona rispetto al memory limit del container
```

💡 **Suggerimento:** Su Java 11+, la JVM rileva automaticamente i cgroup limits del container. Usare `-XX:MaxRAMPercentage=75.0` invece di `-Xmx` fisso: si adatta automaticamente se il memory limit del Deployment cambia.

Verifica Variabili su ARO con oc debug

```
# Debug di un nodo ARO per verificare env vars di sistema
oc debug node/<node-name>

# Debug di un Deployment per verificare le variabili risolte
oc debug deployment/myapp -n myproject -- env | sort
```

Riepilogo dei Punti Chiave

- `env[].value` definisce variabili statiche nel manifest; `env[].valueFrom` legge da ConfigMap, Secret, metadati Pod o valori di risorse
 - `envFrom` importa tutte le chiavi di un ConfigMap o Secret; usare `prefix` per evitare conflitti tra sorgenti diverse
 - **Ordine di precedenza:** `env[].value` > `env[].valueFrom` > `envFrom` (le dichiarazioni più specifiche vincono)
 - `fieldRef` (Downward API) inietta metadati del Pod senza chiamate all'API server: utile per logging strutturato, tracing distribuito, self-registration
 - `resourceFieldRef` espone requests e limits come variabili numeriche: fondamentale per configurare automaticamente la JVM di applicazioni Java senza hardcoding
 - Le chiavi di ConfigMap/Secret con caratteri non validi (`.`, `-`) vengono ignorate silenziosamente da `envFrom`: sempre verificare con `oc exec pod -- env`
-

Domande di Verifica

1. Un container ha `envFrom` con un `ConfigMap` che definisce `LOG_LEVEL=info`, e un `env[]` che definisce `LOG_LEVEL=debug`. Quale valore verrà usato dal container?

- A) `info` (`envFrom` ha priorità)
- B) Entrambi in conflitto causano errore di avvio
- C) `debug` (`env[]` ha priorità su `envFrom`) ✓
- D) Il valore dipende dall'ordine in cui sono dichiarati

2. Quale meccanismo permette di iniettare il nome del nodo su cui gira il Pod come variabile d'ambiente?

- A) `secretKeyRef`
- B) `resourceFieldRef` con `resource: spec.nodeName`
- C) `fieldRef` con `fieldPath: spec.nodeName` ✓
- D) `envFrom` con un `ConfigMap` aggiornato da un `DaemonSet`

3. Un'applicazione Java su ARO deve configurare `-Xmx` in base al memory limit del container (512Mi). Quale approach è più manutenibile?

- A) Hardcodare `-Xmx512m` nella variabile `JAVA_TOOL_OPTIONS`
- B) Usare `resourceFieldRef` con `resource: limits.memory` per ottenere il limite dinamicamente, e calcolare `-Xmx` nello script di avvio ✓
- C) Impostare il memory limit uguale a quello della JVM nel cluster
- D) Non configurare `-Xmx` e lasciare alla JVM la gestione della memoria

4. Quale affermazione su `envFrom` con `secretRef` è corretta?

- A) I valori del Secret vengono mostrati in chiaro in `oc describe pod`
- B) Le chiavi del Secret con caratteri non validi (es. `.`) vengono silenziosamente ignorate ✓
- C) `envFrom.secretRef` supporta la selezione di singole chiavi tramite `items`
- D) I valori vengono aggiornati automaticamente senza riavvio del Pod

5. Quale campo `fieldPath` della Downward API restituisce l'indirizzo IP del Pod?

- A) `spec.podCIDR`
- B) `metadata.podIP`
- C) `status.podIP` ✓
- D) `spec.hostNetwork`